

ACTIVIDAD # 2 - PROGRAMACIÓN ORIENTADA A OBJETOS

Docente: Walter Hugo Arboleda Mazo
Estudiante: Jose David Rangel Sanguino

Universidad Nacional de Colombia - Sede Medellín
Medellín, Antioquia

EJERCICIO 2.1 - CÓDIGO FUENTE

```
public class Persona {
    private String nombre;
    private String apellido;
    private String numeroDocumento;
    private int anoNacimiento;

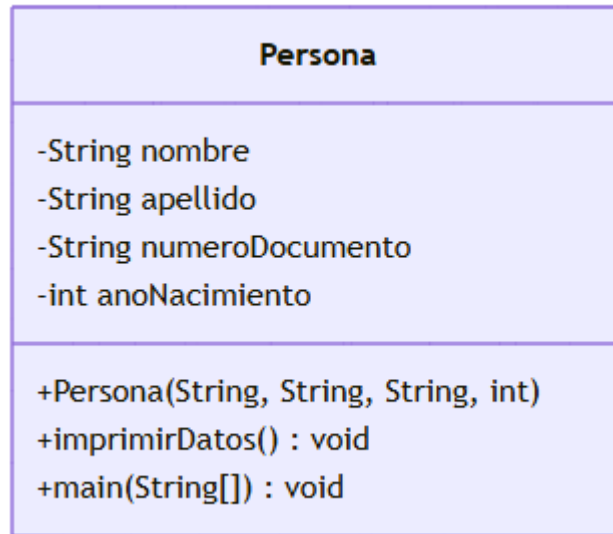
    public Persona(String nombre, String apellido, String numeroDocumento, int
anoNacimiento) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.numeroDocumento = numeroDocumento;
        this.anoNacimiento = anoNacimiento;
    }

    public void imprimirDatos() {
        System.out.println("Nombre: " + nombre);
        System.out.println("Apellido: " + apellido);
        System.out.println("Número de Documento: " + numeroDocumento);
        System.out.println("Año de Nacimiento: " + anoNacimiento);
        System.out.println("-----");
    }

    public static void main(String[] args) {
        Persona persona1 = new Persona("Carlos", "Gómez", "1032456789", 1995);
        Persona persona2 = new Persona("María", "Rodríguez", "9876543210", 2001);

        persona1.imprimirDatos();
        persona2.imprimirDatos();
    }
}
```

EJERCICIO 2.1 DIAGRAMA DE CLASES



EJERCICIO 2.2 CÓDIGO FUENTE

```
public class Planeta {  
  
    public enum TipoPlaneta {  
        GASEOSO, TERRESTRE, ENANO  
    }  
  
    private String nombre = null;  
    private int cantidadSatelites = 0;  
    private double masa = 0;  
    private double volumen = 0;  
    private int diametro = 0;  
    private int distanciaMediaSol = 0;  
    private TipoPlaneta tipoPlaneta;  
    private boolean observableSimpleVista = false;  
  
    public Planeta(String nombre, int cantidadSatelites, double masa, double volumen,  
        int diametro, int distanciaMediaSol, TipoPlaneta tipoPlaneta,  
        boolean observableSimpleVista) {  
        this.nombre = nombre;  
        this.cantidadSatelites = cantidadSatelites;  
        this.masa = masa;  
        this.volumen = volumen;  
        this.diametro = diametro;  
        this.distanciaMediaSol = distanciaMediaSol;
```

```

        this.tipoPlaneta = tipoPlaneta;
        this.observableSimpleVista = observableSimpleVista;
    }

    public double calcularDensidad() {
        if (volumen == 0) {
            return 0;
        }
        return masa / volumen;
    }

    public boolean esPlanetaExterior() {
        double limiteCinturonKM = 3.4 * 149597870.0;
        double distanciaKM = distanciaMediaSol * 1_000_000.0;
        return distanciaKM > limiteCinturonKM;
    }

    public void imprimirAtributos() {
        System.out.println("Nombre: " + nombre);
        System.out.println("Cantidad de satélites: " + cantidadSatelites);
        System.out.println("Masa (kg): " + masa);
        System.out.println("Volumen (km³): " + volumen);
        System.out.println("Diámetro (km): " + diametro);
        System.out.println("Distancia media al Sol (millones km): " + distanciaMediaSol);
        System.out.println("Tipo de planeta: " + tipoPlaneta);
        System.out.println("Observable a simple vista: " + observableSimpleVista);
        System.out.println("Densidad (kg/km³): " + calcularDensidad());
        System.out.println("¿Es planeta exterior?: " + esPlanetaExterior());
        System.out.println("-----");
    }

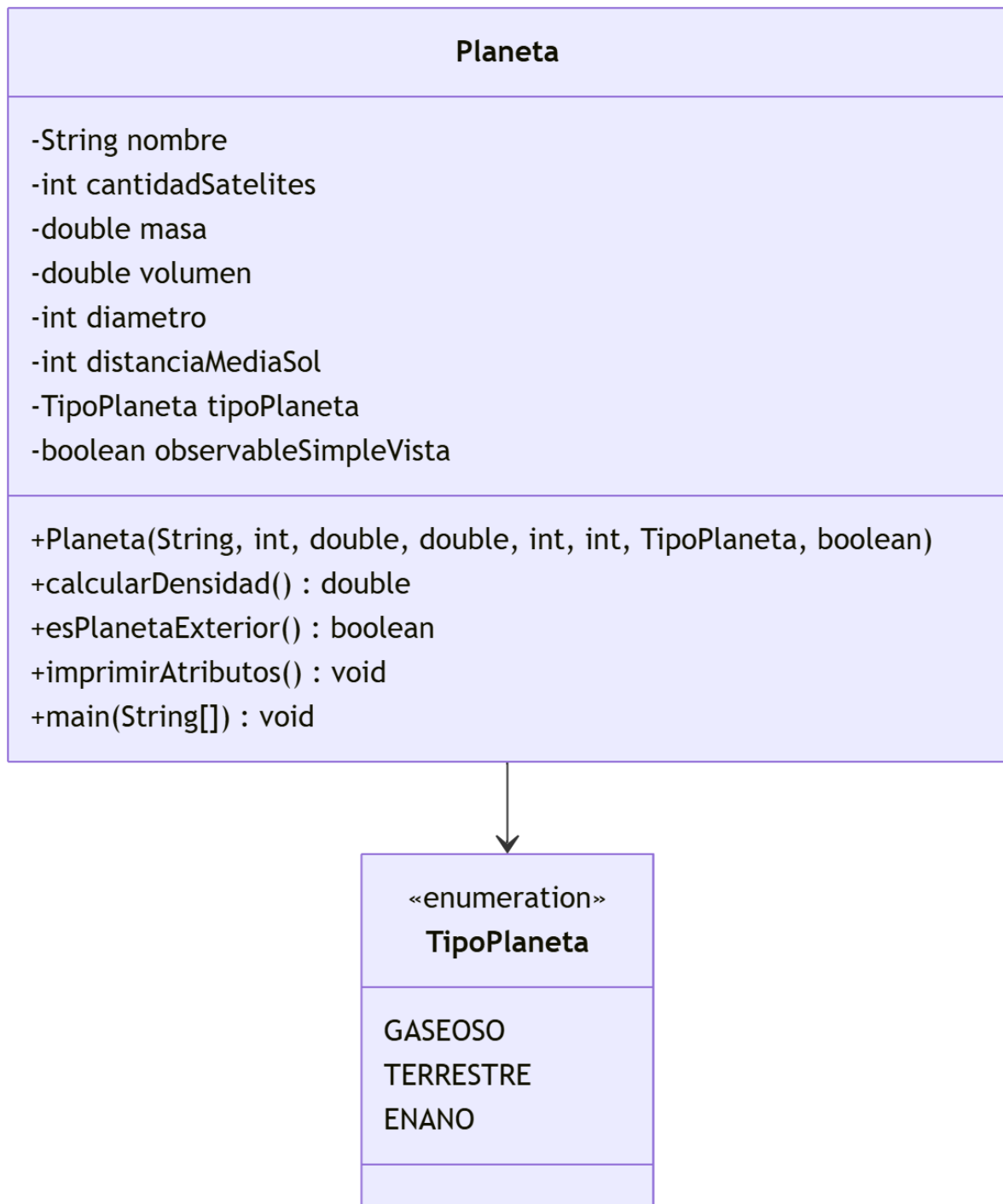
    public static void main(String[] args) {
        Planeta tierra = new Planeta("Tierra", 1, 5.972e24, 1.08321e12,
                                     12742, 149, TipoPlaneta.TERRESTRE, true);

        Planeta jupiter = new Planeta("Júpiter", 95, 1.898e27, 1.43128e15,
                                       139820, 778, TipoPlaneta.GASEOSO, true);

        tierra.imprimirAtributos();
        jupiter.imprimirAtributos();
    }
}

```

EJERCICIO 2.2 DIAGRAMA DE CLASES



EJERCICIO 2.3 CÓDIGO FUENTE

```
public class Automovil {

    public enum TipoCombustible {
        GASOLINA, BIOETANOL, DIESEL, BIODIESEL, GAS_NATURAL
    }

    public enum TipoAutomovil {
        CARRO_DE_CIUDAD, SUBCOMPACTO, COMPACTO, FAMILIAR, EJECUTIVO,
        SUV
    }

    public enum Color {
        BLANCO, NEGRO, ROJO, NARANJA, AMARILLO, VERDE, AZUL, VIOLETA
    }

    private String marca;
    private int modelo;
    private double motor;
    private TipoCombustible tipoCombustible;
    private TipoAutomovil tipoAutomovil;
    private int numeroPuertas;
    private int cantidadAsientos;
    private double velocidadMaxima;
    private Color color;
    private double velocidadActual;

    public Automovil(String marca, int modelo, double motor, TipoCombustible
    tipoCombustible,
        TipoAutomovil tipoAutomovil, int numeroPuertas, int cantidadAsientos,
        double velocidadMaxima, Color color, double velocidadActual) {
        this.marca = marca;
        this.modelo = modelo;
        this.motor = motor;
        this.tipoCombustible = tipoCombustible;
        this.tipoAutomovil = tipoAutomovil;
        this.numeroPuertas = numeroPuertas;
        this.cantidadAsientos = cantidadAsientos;
        this.velocidadMaxima = velocidadMaxima;
        this.color = color;
        this.velocidadActual = velocidadActual;
    }
}
```

```
public String getMarca() {  
    return marca;  
}
```

```
public void setMarca(String marca) {  
    this.marca = marca;  
}
```

```
public int getModelo() {  
    return modelo;  
}
```

```
public void setModelo(int modelo) {  
    this.modelo = modelo;  
}
```

```
public double getMotor() {  
    return motor;  
}
```

```
public void setMotor(double motor) {  
    this.motor = motor;  
}
```

```
public TipoCombustible getTipoCombustible() {  
    return tipoCombustible;  
}
```

```
public void setTipoCombustible(TipoCombustible tipoCombustible) {  
    this.tipoCombustible = tipoCombustible;  
}
```

```
public TipoAutomovil getTipoAutomovil() {  
    return tipoAutomovil;  
}
```

```
public void setTipoAutomovil(TipoAutomovil tipoAutomovil) {  
    this.tipoAutomovil = tipoAutomovil;  
}
```

```
public int getNumeroPuertas() {  
    return numeroPuertas;  
}
```

```

public void setNumeroPuertas(int numeroPuertas) {
    this.numeroPuertas = numeroPuertas;
}

public int getCantidadAsientos() {
    return cantidadAsientos;
}

public void setCantidadAsientos(int cantidadAsientos) {
    this.cantidadAsientos = cantidadAsientos;
}

public double getVelocidadMaxima() {
    return velocidadMaxima;
}

public void setVelocidadMaxima(double velocidadMaxima) {
    this.velocidadMaxima = velocidadMaxima;
}

public Color getColor() {
    return color;
}

public void setColor(Color color) {
    this.color = color;
}

public double getVelocidadActual() {
    return velocidadActual;
}

public void setVelocidadActual(double velocidadActual) {
    this.velocidadActual = velocidadActual;
}

public void acelerar(double incremento) {
    if (velocidadActual + incremento > velocidadMaxima) {
        System.out.println("No se puede acelerar más allá de la velocidad máxima permitida (" + velocidadMaxima + " km/h).");
    } else {
        velocidadActual += incremento;
    }
}

```

```

public void desacelerar(double decremento) {
    if (velocidadActual - decremento < 0) {
        System.out.println("No es posible desacelerar a una velocidad negativa.");
    } else {
        velocidadActual -= decremento;
    }
}

public void frenar() {
    velocidadActual = 0;
}

public double calcularTiempoEstimadoLlegada(double distancia) {
    if (velocidadActual == 0) {
        return 0;
    }
    return distancia / velocidadActual;
}

public void imprimirAtributos() {
    System.out.println("Marca: " + marca);
    System.out.println("Modelo: " + modelo);
    System.out.println("Motor (L): " + motor);
    System.out.println("Tipo de Combustible: " + tipoCombustible);
    System.out.println("Tipo de Automóvil: " + tipoAutomovil);
    System.out.println("Número de Puertas: " + numeroPuertas);
    System.out.println("Cantidad de Asientos: " + cantidadAsientos);
    System.out.println("Velocidad Máxima: " + velocidadMaxima + " km/h");
    System.out.println("Color: " + color);
    System.out.println("Velocidad Actual: " + velocidadActual + " km/h");
    System.out.println("-----");
}

public static void main(String[] args) {
    Automovil auto = new Automovil("Toyota", 2023, 2.0, TipoCombustible.GASOLINA,
                                    TipoAutomovil.COMPACTO, 4, 5, 180, Color.AZUL, 0);

    auto.imprimirAtributos();

    auto.setVelocidadActual(100);
    System.out.println("Velocidad actual: " + auto.getVelocidadActual() + " km/h");

    auto.acelerar(20);
}

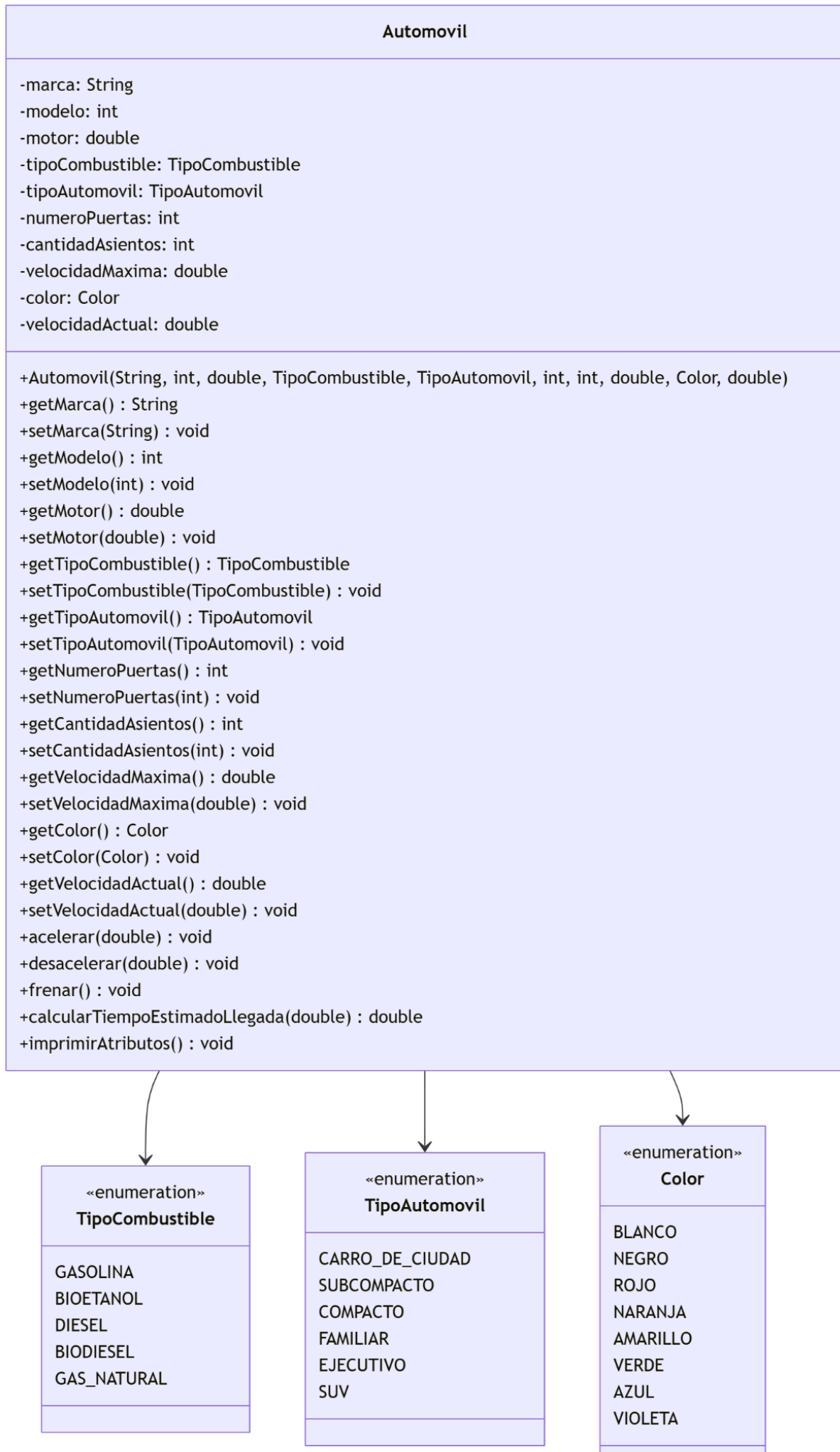
```

```
        System.out.println("Velocidad actual tras acelerar 20 km/h: " +
auto.getVelocidadActual() + " km/h");

        auto.desacelerar(50);
        System.out.println("Velocidad actual tras desacelerar 50 km/h: " +
auto.getVelocidadActual() + " km/h");

        auto.frenar();
        System.out.println("Velocidad actual tras frenar: " + auto.getVelocidadActual() + "
km/h");
    }
}
```

EJERCICIO 2.3 DIAGRAMA DE CLASES (PÁGINA DEBAJO)



EJERCICIO 2.4 CODIGO FUENTE

```
import math
class Circulo {
    int radio;

    public Circulo(int radio) {
        this.radio = radio;
    }

    public double calcularArea() {
        return Math.PI * Math.pow(radio, 2);
    }

    public double calcularPerimetro() {
        return 2 * Math.PI * radio;
    }
}

class Rectangulo {
    int base;
    int altura;

    public Rectangulo(int base, int altura) {
        this.base = base;
        this.altura = altura;
    }

    public double calcularArea() {
        return base * altura;
    }

    public double calcularPerimetro() {
        return (2 * base) + (2 * altura);
    }
}

class Cuadrado {
    int lado;

    public Cuadrado(int lado) {
        this.lado = lado;
    }
}
```

```

    public double calcularArea() {
        return lado * lado;
    }

    public double calcularPerimetro() {
        return 4 * lado;
    }
}

class TrianguloRectangulo {
    int base;
    int altura;

    public TrianguloRectangulo(int base, int altura) {
        this.base = base;
        this.altura = altura;
    }

    public double calcularArea() {
        return (base * altura) / 2.0;
    }

    public double calcularHipotenusa() {
        return Math.pow(Math.pow(base, 2) + Math.pow(altura, 2), 0.5);
    }

    public double calcularPerimetro() {
        return base + altura + calcularHipotenusa();
    }

    public void determinarTipoTriangulo() {
        double hipotenusa = calcularHipotenusa();

        if ((base == altura) && (base == hipotenusa) && (altura == hipotenusa)) {
            System.out.println("Es un triángulo equilátero");
        } else if ((base != altura) && (base != hipotenusa) && (altura != hipotenusa)) {
            System.out.println("Es un triángulo escaleno");
        } else {
            System.out.println("Es un triángulo isósceles");
        }
    }
}

public class PruebaFiguras {

```

```

public static void main(String[] args) {
    Circulo figura1 = new Circulo(2);
    Rectangulo figura2 = new Rectangulo(1, 2);
    Cuadrado figura3 = new Cuadrado(3);
    TrianguloRectangulo figura4 = new TrianguloRectangulo(3, 5);

    System.out.println("El área del círculo es = " + figura1.calcularArea());
    System.out.println("El perímetro del círculo es = " + figura1.calcularPerimetro());
    System.out.println();

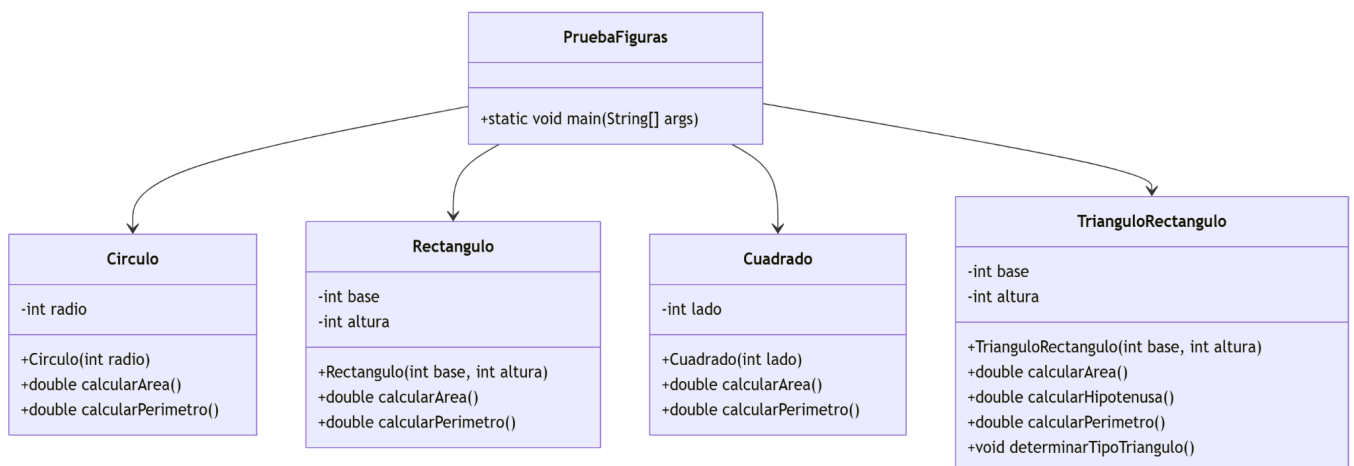
    System.out.println("El área del rectángulo es = " + figura2.calcularArea());
    System.out.println("El perímetro del rectángulo es = " + figura2.calcularPerimetro());
    System.out.println();

    System.out.println("El área del cuadrado es = " + figura3.calcularArea());
    System.out.println("El perímetro del cuadrado es = " + figura3.calcularPerimetro());
    System.out.println();

    System.out.println("El área del triángulo es = " + figura4.calcularArea());
    System.out.println("El perímetro del triángulo es = " + figura4.calcularPerimetro());
    figura4.determinarTipoTriangulo();
}
}

```

EJERCICIO 2.4 DIAGRAMA DE CLASES



EJERCICIO 2.5 CÓDIGO FUENTE

```
public class CuentaBancaria {

    public enum TipoCuenta {
        AHORROS, CORRIENTE
    }

    private String nombresTitular;
    private String apellidosTitular;
    private int numeroCuenta;
    private TipoCuenta tipoCuenta;
    private float saldo = 0;

    public CuentaBancaria(String nombresTitular, String apellidosTitular, int numeroCuenta,
TipoCuenta tipoCuenta) {
        this.nombresTitular = nombresTitular;
        this.apellidosTitular = apellidosTitular;
        this.numeroCuenta = numeroCuenta;
        this.tipoCuenta = tipoCuenta;
        this.saldo = 0;
    }

    public void imprimirAtributos() {
        System.out.println("Nombres del titular: " + nombresTitular);
        System.out.println("Apellidos del titular: " + apellidosTitular);
        System.out.println("Número de cuenta: " + numeroCuenta);
        System.out.println("Tipo de cuenta: " + tipoCuenta);
        System.out.println("Saldo de la cuenta: $" + saldo);
        System.out.println("-----");
    }

    public float consultarSaldo() {
        return saldo;
    }

    public void consignar(float valor) {
        if (valor > 0) {
            saldo += valor;
            System.out.println("Se han consignado $" + valor + ". Nuevo saldo: $" + saldo);
        } else {
            System.out.println("El valor a consignar debe ser mayor a cero.");
        }
    }
}
```

```

public void retirar(float valor) {
    if (valor > 0 && valor <= saldo) {
        saldo -= valor;
        System.out.println("Se han retirado $" + valor + ". Nuevo saldo: $" + saldo);
    } else if (valor > saldo) {
        System.out.println("El valor a retirar supera el saldo actual de la cuenta.");
    } else {
        System.out.println("El valor a retirar debe ser mayor a cero.");
    }
}
}

```

```

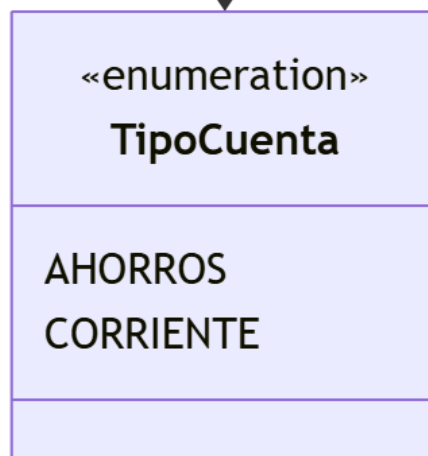
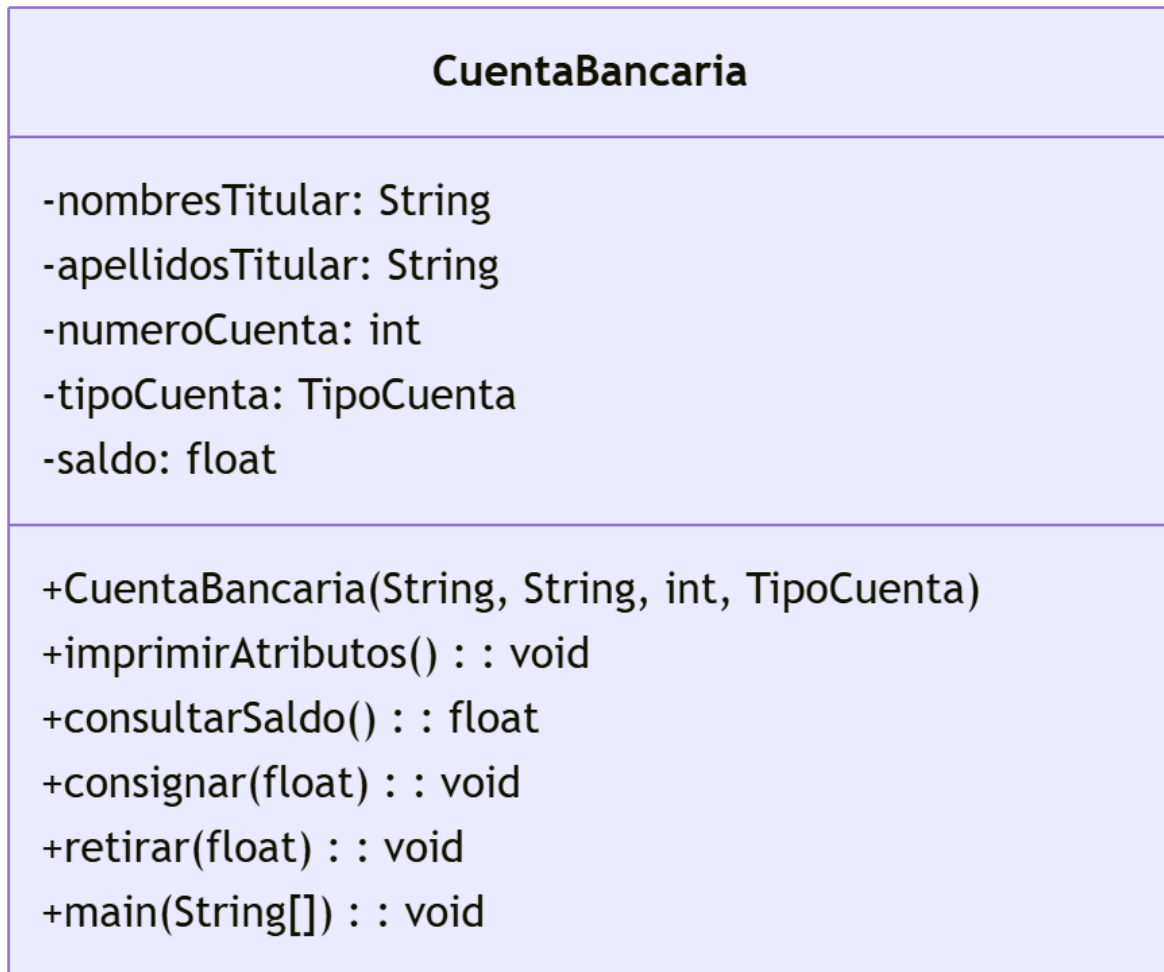
public static void main(String[] args) {
    CuentaBancaria cuenta = new CuentaBancaria("Pedro", "Pérez", 12345678,
TipoCuenta.AHORROS);

    cuenta.imprimirAtributos();

    cuenta.consignar(100000);
    cuenta.consignar(300000

```

EJERCICIO 2.5 DIAGRAMA DE CLASES



CUENTA GITHUB: [GitHub - jorangels-lab/Python-OOP-UNAL · GitHub](https://github.com/jorangels-lab/Python-OOP-UNAL)